Activity No. 13	
Working with Files and Directories in Linux	
Course Code: CPE 201A	**Program:BS Computer Engineering**
Course Title: COMPUTER SYSTEM ADMINISTRATION AND TROUBLESHOOTING	**Date Performed: Oct 20, 2026**
Section:CPE11S2	**Date Submitted:**
Name: Jose, Redge Jansen A.	**Instructor: Sir Noel Jayson Lusung**

1. Objective/s:

This activity aims to navigate and manage files and directories using command line interface.

2. Intended Learning Outcome/s:

The students should be able to:

2.1 Demonstrate how to navigate home and system directories
2.2 Demonstrate how to list files and directories.
2.3 Demonstrate how to use globbing to manipulate files and directories.
2.4 Demonstrate how to create, move and delete files and directories.

3. Discussion:

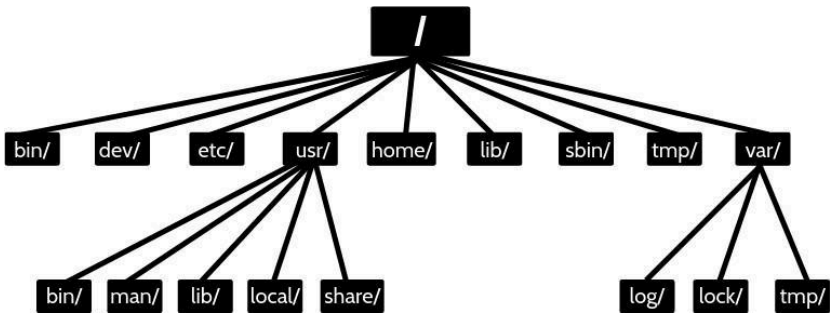
Files and Directories

The Linux filesystem is similar to other operating system's filesystems in that it contains *files* and *directories*. Files contain data such as human-readable text, executable programs, or binary data that is used by the computer.

Directories are used to create organization within the filesystem. Directories can contain files and other directories.

Directory Structure

On a Windows system, the *top level* of the directory structure is called ***My Computer.*** The Linux directory structure, called a ***filesystem***, also has a top level called the ***root directory*** (symbolized by the **slash /character**).



Home Directory

On most Linux distributions there is a directory called **home** under the root /directory. Under this **/home** directory there is a directory for each user on the system. When a user opens a shell, they should

automatically be placed in their home directory. The user has the full control to create and delete additional files and directories in their home directory. Most other directories in a Linux filesystem are protected with *file permissions*. The home directory has a special symbol used to represent it, the **tilde ~ character**.

The directory name is the same as the name of the user. So, a user named sysadmin would have a *home directory* called /home/sysadmin:

File and Directory Names

File and directory names in Linux can contain lower case and upper case letters, numbers, spaces and special characters. However, since many special characters have a special meaning in the Linux shell, it is good practice to not use spaces or special characters when naming files or directories. Spaces, for example, need the *escape character* \ to be entered correctly

Paths

A path is a list of directories separated by the / character. There are two types of paths: *absolute* and *relative*. For example, /home/sysadmin is a path to the home directory

A. Absolute Paths

Absolute paths allow the user to specify the exact location of a directory. Absolute paths always starts at the root directory, and therefore it always begins with the / character.

B. Relative Paths

A relative path gives directions to a file relative to the current location in the filesystem. The user must currently be in a directory that contains objects in the path. Relative paths start with the name of a directory.

Special Relative Paths

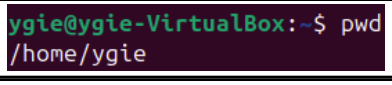
The single period . character always represents the current directory.
Two period .. characters always represents one directory higher relative to the current directory, sometimes referred to as the parent directory.

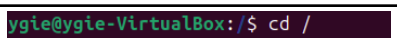
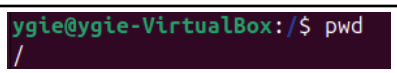
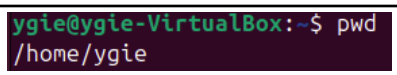
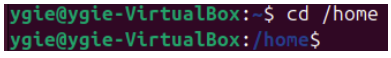
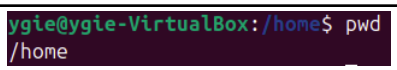
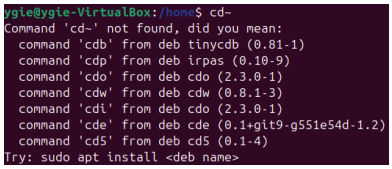
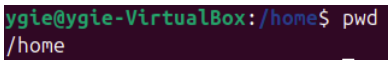
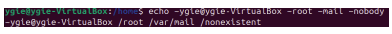
4. Resources:

Personal Computer with installed Virtual Box
Ubuntu Server or Desktop virtual machine

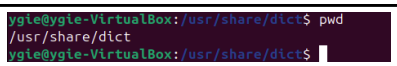
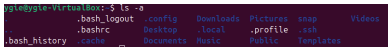
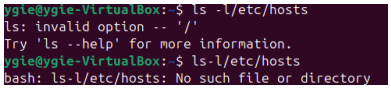
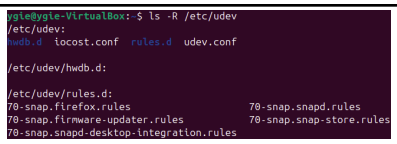
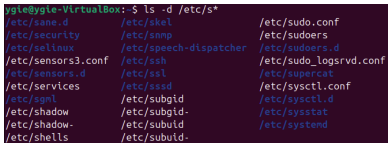
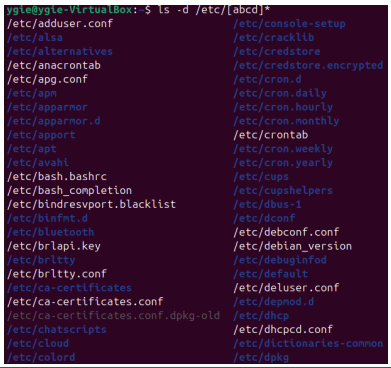
5. Procedure:

1. Login using your username and password.
2. Use terminal emulator application (if you are using desktop version)
3. Execute the following commands to navigate files and directories. Copy a screenshot as output after you execute the given command. Create a brief explanation of the command.

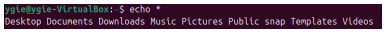
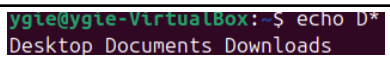
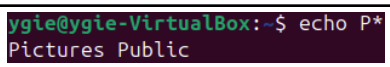
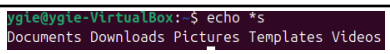
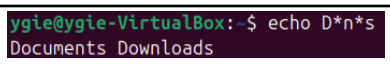
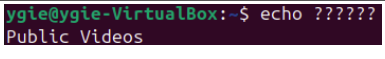
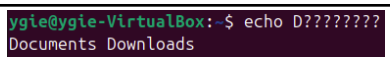
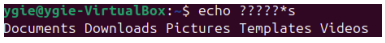
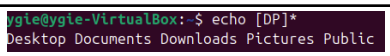
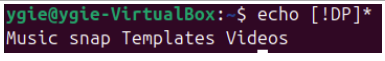
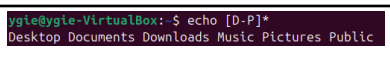
Command	Screenshot	Explanation
1. pwd		Prints the working directory, the

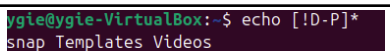
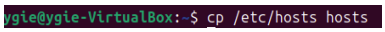
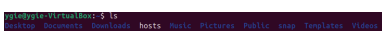
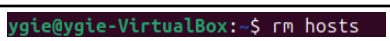
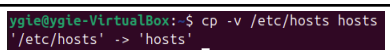
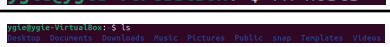
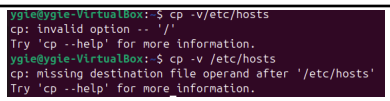
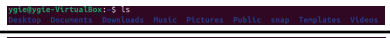
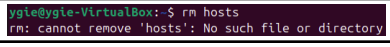
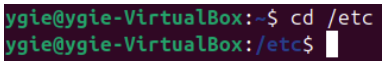
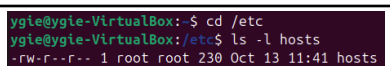
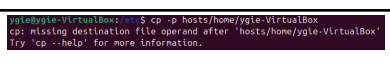
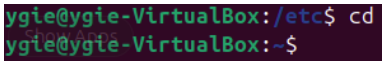
		full path of where you currently are. A new shell starts in your home directory, so it shows /home/<username>.
2. cd /		Changes the directory to the root directory (/), the top level of the Linux filesystem. This is an absolute path.
3. pwd		Confirms the move. The output is /.
4. cd		Typing cd with no argument returns to your home directory from anywhere.
5. pwd		Shows /home/<username>, confirming cd without arguments went home.
6. cd /home		Shows /home/<username>, confirming cd without arguments went home.
7. pwd		Output is /home.
8. cd ~		The tilde (~) is a shortcut for your home directory, so this goes back to /home/<username>.
9. pwd		Output is /home/<username>.
10. echo ~ ~sysadmin ~root ~mail ~nobody where sysadmin is your username		Tilde expansion: the shell replaces each ~ with a home directory path before echo prints it. ~ becomes your own home (/home/<username>); ~sysadmin becomes /home/sysadmin; ~root becomes /root. ~mail and ~nobody become

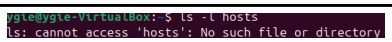
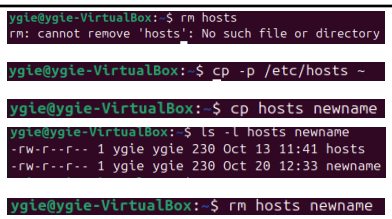
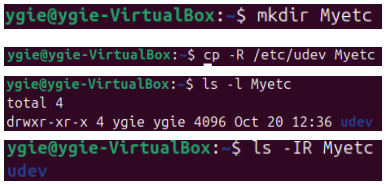
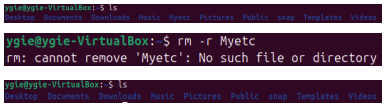
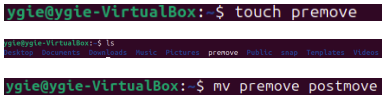
		those system accounts' home directories (usually /var/mail and /nonexistent). A name that is not a real user is printed unchanged.
11. cd ~root	ygie@ygie-VirtualBox:/home$ cd ~root bash: cd: /root: Permission denied	Tries to enter root's home directory (/root). A normal user gets 'Permission denied' because /root is protected by file permissions.
12. cd /usr/bin	ygie@ygie-VirtualBox:/home$ cd /usr/bin ygie@ygie-VirtualBox:/usr/bin$	Changes to /usr/bin, where most executable programs are stored. Uses an absolute path.
13. pwd	ygie@ygie-VirtualBox:/usr/bin$ pwd /usr/bin	Output is /usr/bin.
14. cd /usr	ygie@ygie-VirtualBox:/usr/bin$ cd /usr ygie@ygie-VirtualBox:/usr$	Moves to /usr using an absolute path.
15. type ls	ygie@ygie-VirtualBox:/usr$ type ls ls is aliased to `ls --color=auto'	Shows how the shell interprets 'ls': an alias (ls --color=auto) or the file path of the program, such as /usr/bin/ls.
16. pwd	ygie@ygie-VirtualBox:/usr$ pwd /usr	Output is /usr. type does not change the directory.
17. cd /usr/share/doc	ygie@ygie-VirtualBox:/usr$ cd /usr/share/doc ygie@ygie-VirtualBox:/usr/share/doc$	Moves to /usr/share/doc, which holds documentation for installed packages.
18. pwd	ygie@ygie-VirtualBox:/usr/share/doc$ pwd /usr/share/doc	Output is /usr/share/doc.
19. cd bash	ygie@ygie-VirtualBox:/usr/share/doc$ cd bash ygie@ygie-VirtualBox:/usr/share/doc/bash$	Relative path: enters the bash directory inside the current directory (/usr/share/doc/bash).
20. pwd	ygie@ygie-VirtualBox:/usr/share/doc/bash$ pwd /usr/share/doc/bash	Output is /usr/share/doc/bash.
21. cd ..	ygie@ygie-VirtualBox:/usr/share/doc/bash$ cd .. ygie@ygie-VirtualBox:/usr/share/doc$	The .. means the parent directory, so this moves up one level to /usr/share/doc.
22. pwd	ygie@ygie-VirtualBox:/usr/share/doc$ pwd /usr/share/doc	Output is /usr/share/doc.
23. cd ../dict	ygie@ygie-VirtualBox:/usr/share/doc$ cd ../dict ygie@ygie-VirtualBox:/usr/share/dict$	Goes up one level with .. and then down into dict, so the result is /usr/share/dict. If the dictionary package is not installed, an error 'No such file or directory' appears.

24. pwd		Output is /usr/share/dict (or /usr/share/doc if the previous command failed).
25. cd		Returns to the home directory.
26. ls		Lists the visible (non-hidden) files and directories in the home directory, such as Desktop, Documents, Downloads, Music, Pictures, Public, Templates, Videos.
27. ls -a		-a shows all files, including hidden ones whose names begin with a dot (.bashrc, .profile, .config, and the entries . and ..).
28. ls -l /etc/hosts		-l gives a long listing for /etc/hosts: permissions (-rw-r--r--), link count, owner (root), group (root), size, modification date and name.
29. ls -R /etc/udev		-R lists recursively: the contents of /etc/udev and of every subdirectory under it (such as rules.d).
30. ls -d /etc/s*		The * wildcard matches any characters, so /etc/s* matches everything in /etc starting with s. -d lists directories themselves rather than their contents.
31. ls -d /etc/????		Each ? matches exactly one character, so this lists names in /etc that are exactly four characters long. -d shows directories by name only.
32. ls –d /etc/[abcd]*		[abcd] matches one character from the set, so this lists items in /etc whose names start with a, b, c or d. (The lab text has an en dash in '–d'; type a normal hyphen.)

4. Execute the following commands to manage files and directories. Copy a screenshot as output after you execute the given command. Create a brief explanation of the command.

Command	Screenshot	Explanation
1. echo *		The shell expands * to every non-hidden file and directory name in the current directory, and echo prints them (e.g. Desktop Documents Downloads Music Pictures Public Templates Videos).
2. echo D*		D* matches names that start with D: Desktop Documents Downloads.
3. echo P*		P* matches names that start with P: Pictures Public.
4. echo *s		*s matches names that end with s: Documents Downloads Templates Videos.
5. echo D*n*s		Matches names that start with D, contain an n somewhere, and end with s: Documents Downloads.
6. echo ??????		Each ? is one character, so this matches names exactly six characters long: Public Videos.
7. echo D????????		D followed by eight ? matches names exactly nine characters long that start with D: Documents Downloads.
8. echo ?????*s		Five ? then * then s matches names of at least six characters that end in s: Documents Downloads Templates Videos.
9. echo [DP]*		[DP] matches one character, D or P, so names beginning with D or P: Desktop Documents Downloads Pictures Public.
10. echo [!DP]*		[!DP] negates the set: names that do not begin with D or P: Music Templates Videos.
11. echo [D-P]*		[D-P] is a range of letters D through P: Desktop Documents

		Downloads Music Pictures Public.
12. echo [!D-P]*		[!D-P] negates the range: names not starting with a letter from D to P: Templates Videos.
13. ls		Lists the current directory. At this point there is no file named hosts yet.
14. cp /etc/hosts hosts		Copies /etc/hosts into the current directory under the name hosts. No output on success.
15. ls		hosts now appears in the listing, confirming the copy worked.
16. rm hosts		Deletes the copied hosts file. No output; rm does not use a recycle bin.
17. ls		hosts is gone from the listing.
18. cp –v /etc/hosts hosts		Same copy, but -v (verbose) prints what was done: '/etc/hosts' -> 'hosts'.
19. ls		hosts appears again.
20. rm hosts		Removes the file again.
21. ls		hosts is no longer listed.
22. cp –v /etc/hosts .		Copies /etc/hosts into the current directory (the . destination), keeping its name, and prints the action because of -v.
23. ls		hosts appears in the listing.
24. rm hosts		Removes the copy.
25. ls		hosts is gone again.
26. cd /etc		Changes to the /etc directory.
27. ls -l hosts		Long listing of the original /etc/hosts: it shows owner root and the original modification date and time.
28. cp –p hosts/home/sysadmin		-p preserves attributes (mode, timestamps, and ownership if allowed) when copying /etc/hosts to /home/sysadmin.
29. cd		Returns to the home directory.

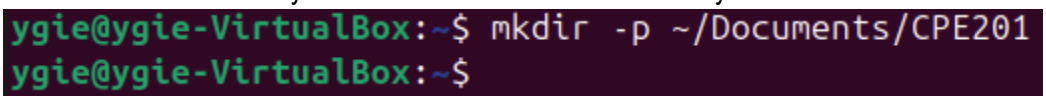
30. ls –l hosts		Lists the copy. Its date and time match the original in /etc because -p preserved them; an owner other than root is normal for a non-root user.
31. rm hosts 32. cp -p /etc/hosts ~ 33. cp hosts newname 34. ls –l hosts newname 35. rm hosts newname		Deletes the copy in the home directory. Copies /etc/hosts to the home directory (~), preserving attributes. Makes a copy named newname from hosts without -p, so it gets the current date/time and is owned by you. Compares both files: hosts keeps the original timestamp, newname shows the time of the copy. Deletes both files at once.
36. mkdir Myetc 37. cp –R /etc/udev Myetc 38. ls –l Myetc 39. ls –lR Myetc		Creates a new empty directory called Myetc. -R copies /etc/udev and everything inside it recursively into Myetc, creating Myetc/udev. Without -R, cp refuses to copy a directory. Shows the contents of Myetc (the udev directory) in long format. -lR shows a recursive long listing of Myetc and all its subdirectories and files.
40. ls 41. rm -r Myetc 42. ls		Lists the home directory, where Myetc is still present. rm -r deletes Myetc and everything inside it recursively. (In the lab sheet this line and the next run together: 'rm -r Myetc' then 'ls'.) The following ls confirms that Myetc is gone.
43. touch premove 44. ls 45. mv premove postmove		touch creates an empty file named premove (or updates its timestamp if it exists).

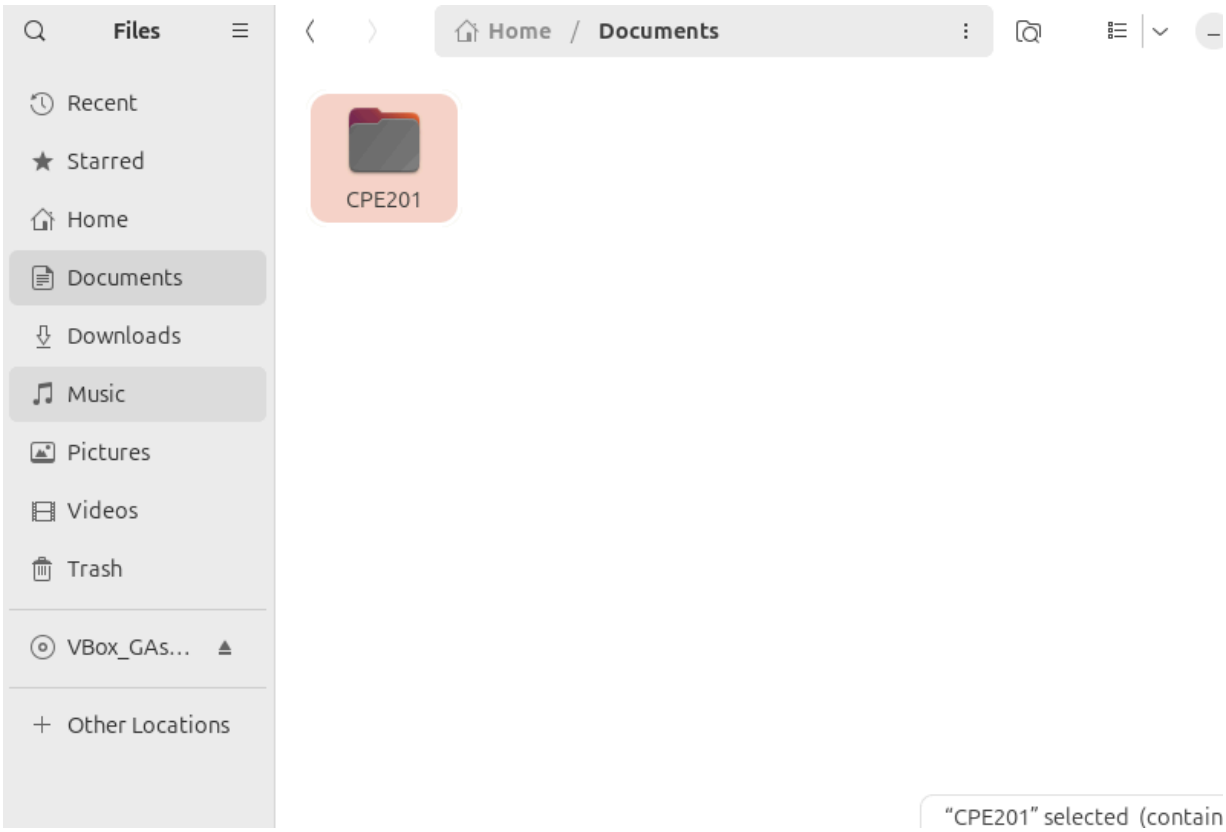
46. ls 47. rm postmove		premove appears in the listing. mv renames premove to postmove (moving a file within the same directory is a rename). Only postmove is listed; premove no longer exists. Deletes postmove, cleaning up.

6. Supplementary Activity:

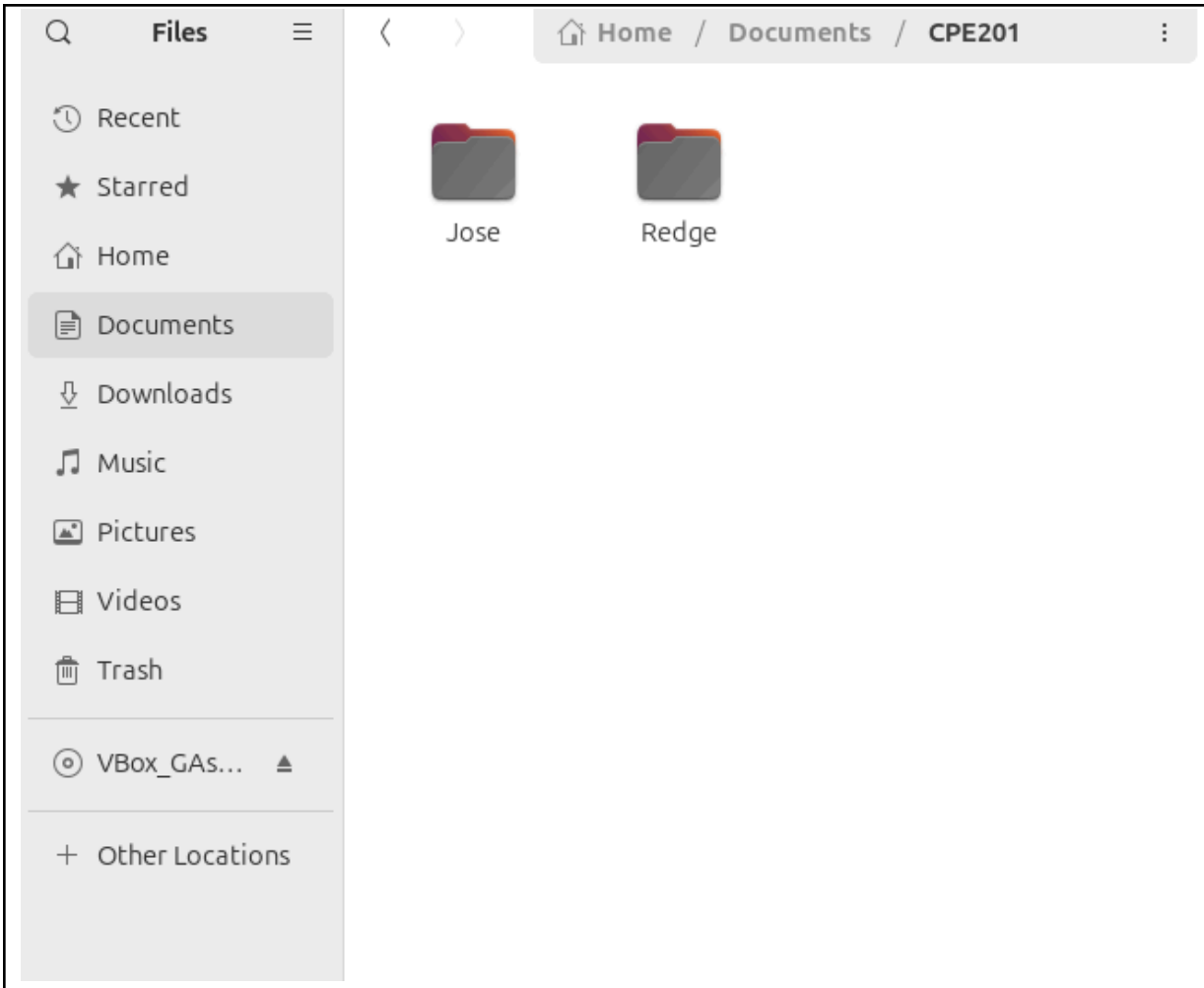
Copy screen shot(s) of the following tasks:

1. Create a directory CPE201 in the Documents directory.

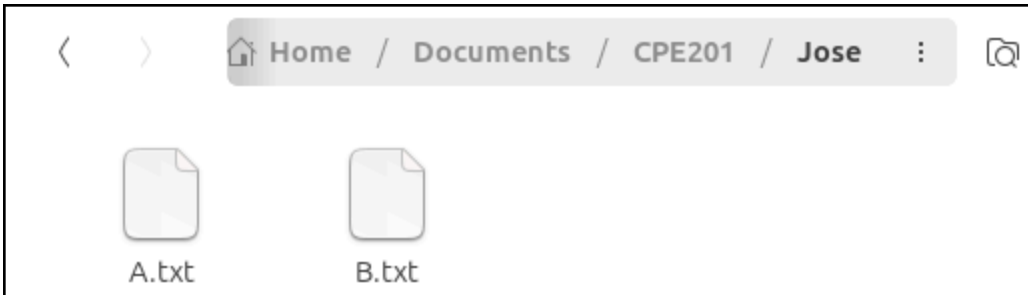




2. Create a directory <lastname> and <firstname> under CPE201 directory.

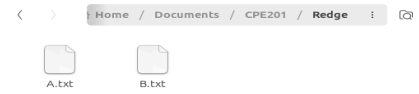


3. Create two text files: (1) A.txt and (2) B.txt in the <lastname> directory.



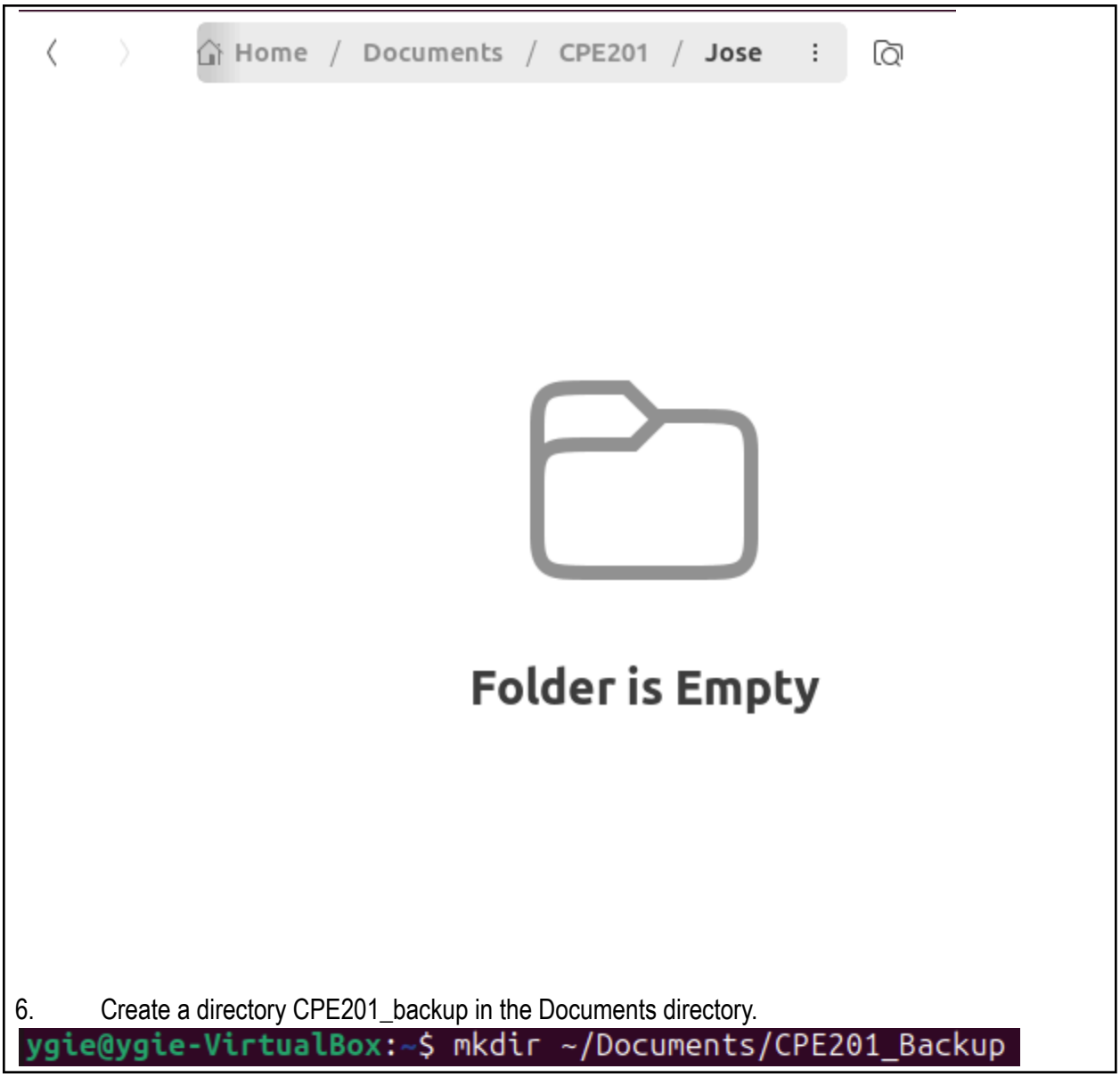
4. Copy the content of <lastname> directory to <firstname> directory.

```
ygie@ygie-VirtualBox:~$ cp -r ~/Documents/CPE201/Jose/. ~/Documents/CPE201/Redge
```



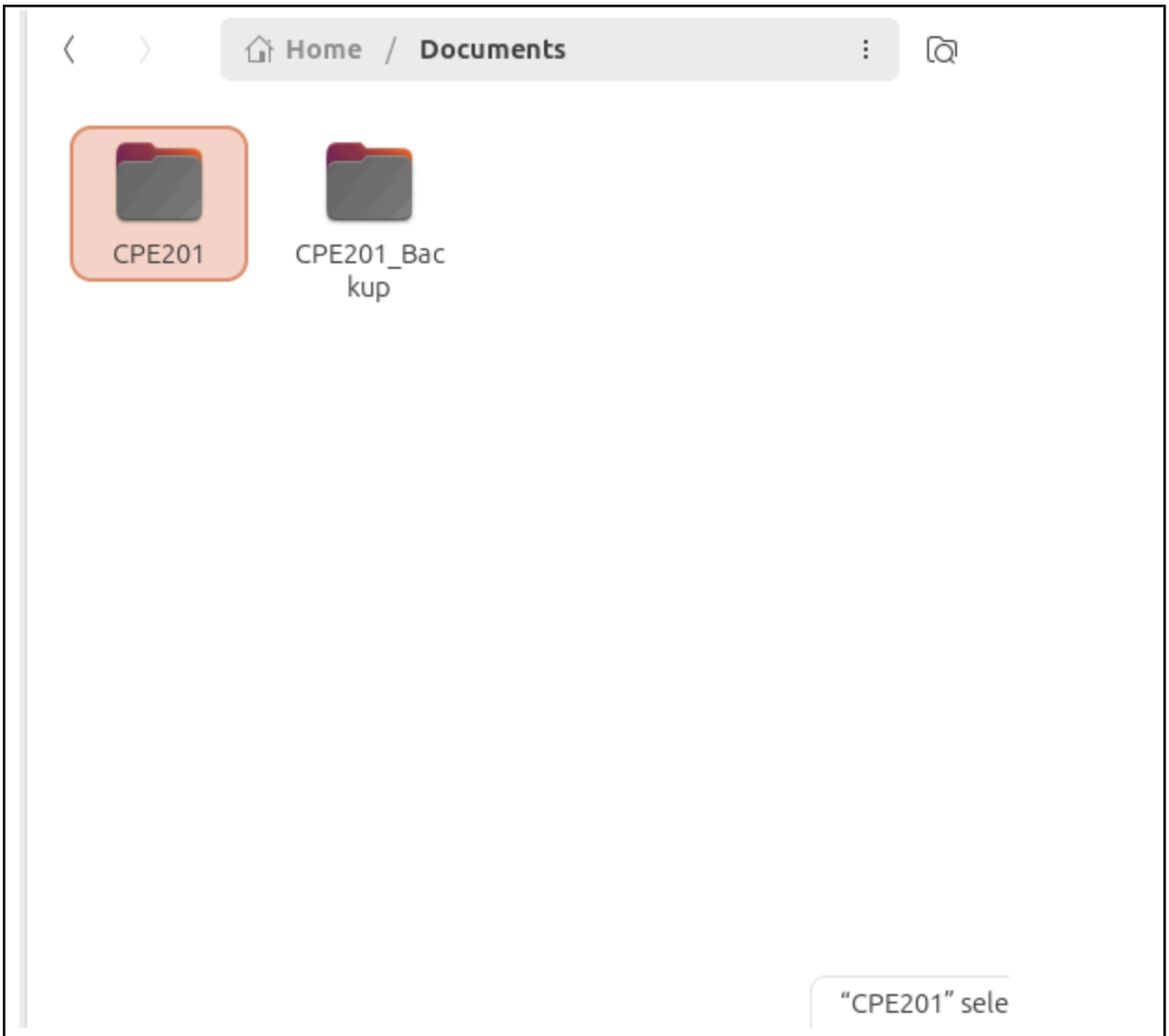
5. Delete the content of <lastname> directory.

```
ygie@ygie-VirtualBox:~$ rm -rf ~/Documents/CPE201/Jose/*
```



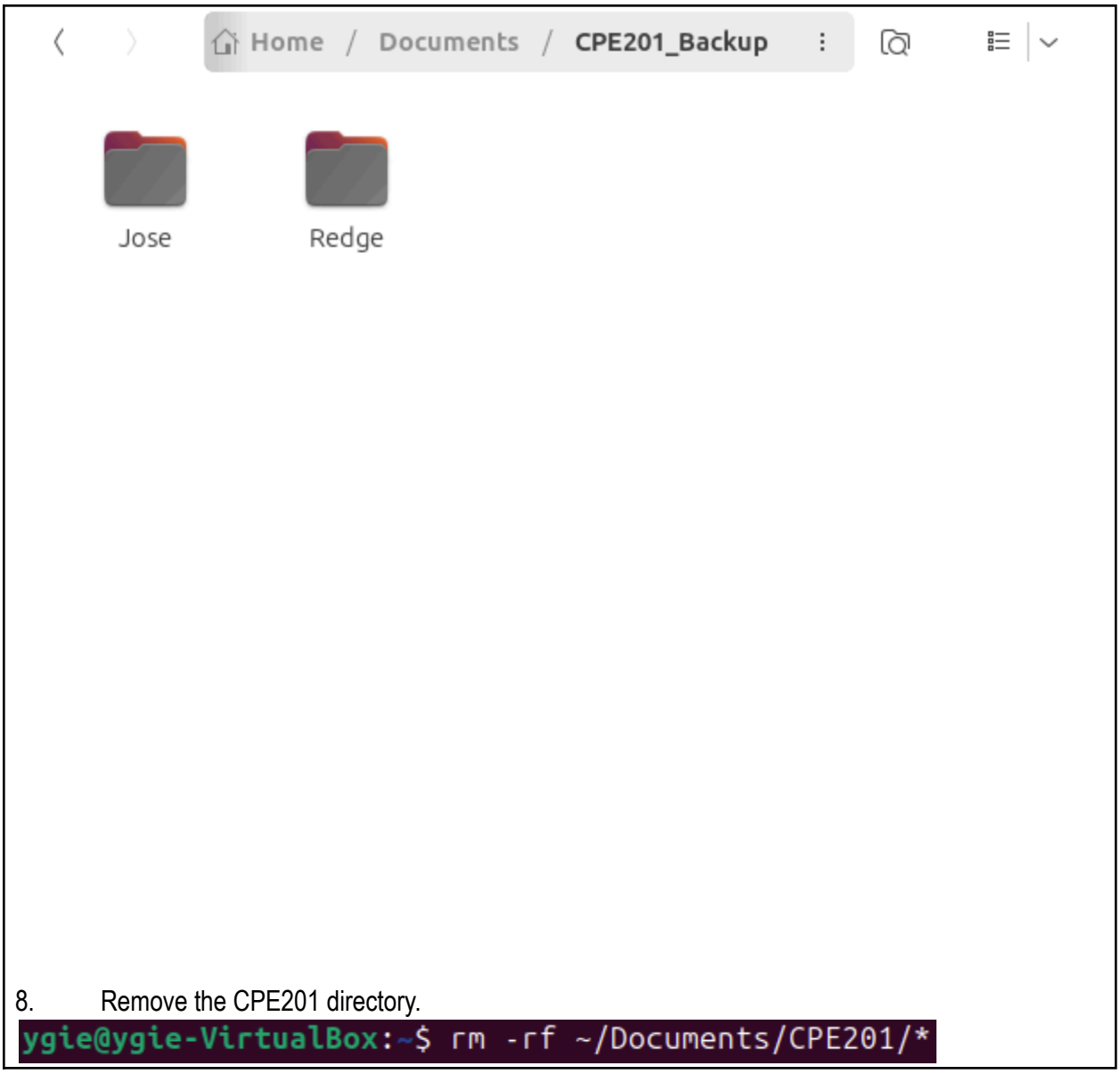
6. Create a directory CPE201_backup in the Documents directory.

```
ygie@ygie-VirtualBox:~$ mkdir ~/Documents/CPE201_Backup
```



7. Copy the content of CPE201 to CPE201_backup directory.

```
ygie@ygie-VirtualBox:~$ cp -r ~/Documents/CPE201/. ~/Documents/CPE201_Backup
```



8. Remove the CPE201 directory.

```
ygie@ygie-VirtualBox:~$ rm -rf ~/Documents/CPE201/*
```



7. Conclusion:

In this activity, I learned the core Linux CLI skills required for everyday system administration. I got comfortable moving around the filesystem with pwd and cd, taking advantage of path shortcuts like ~ and .. Using ls with various flags (-a, -l, -R, -d) and globbing patterns allowed me to efficiently view and target specific files. I also practiced creating, copying, moving, renaming, and deleting files and directories with tools like mkdir, touch, cp, mv, and rm. Overall, it gave me a practical understanding of basic file management, the importance of syntax accuracy with permanent commands like rm, and how Linux permissions keep sensitive system locations like /root protected.

8. Assessment (Rubric for Laboratory Performance):